Windows Services

Architecture, Development & Deployment

A Beginner's Complete Guide to .NET Windows Services

Topics Covered

- Architecture & Components
- Creating a Windows Service in .NET
- Installing & Managing Services
- Debugging Windows Services
- Advanced Service Features

Audience: Freshers / Beginners

1. Introduction to Windows Services Architecture & Components

What is a Windows Service?

A Windows Service is a long-running executable that performs specific functions and which can be automatically started when the computer boots, paused, and stopped. Unlike desktop applications, services run in the background without a user interface.

Think of services like security guards — they are always running in the background, even when no one is actively using the computer. Examples you already use every day: Windows Update, antivirus software, and print spoolers are all Windows Services.

Windows Service program that runs in the **background** without a visible window. It starts automatically with Windows and keeps running even when no user is logged in — like a web server, antivirus, or database engine.

Key Points:

- Runs in **background**
- Does **not require user interaction**
- Operates under **Service Control Manager (SCM)**

Real world Examples

1. SQL Sever Service
2. IIS
3. Windows Update Service
4. Message Queue processors (RabbitMQ consumers)

> **Key Difference**
> A normal application needs a user to open it. A Windows Service is managed by the operating system — it can auto-start on boot, restart on crash, and run under a special account with no desktop.

Key Characteristics

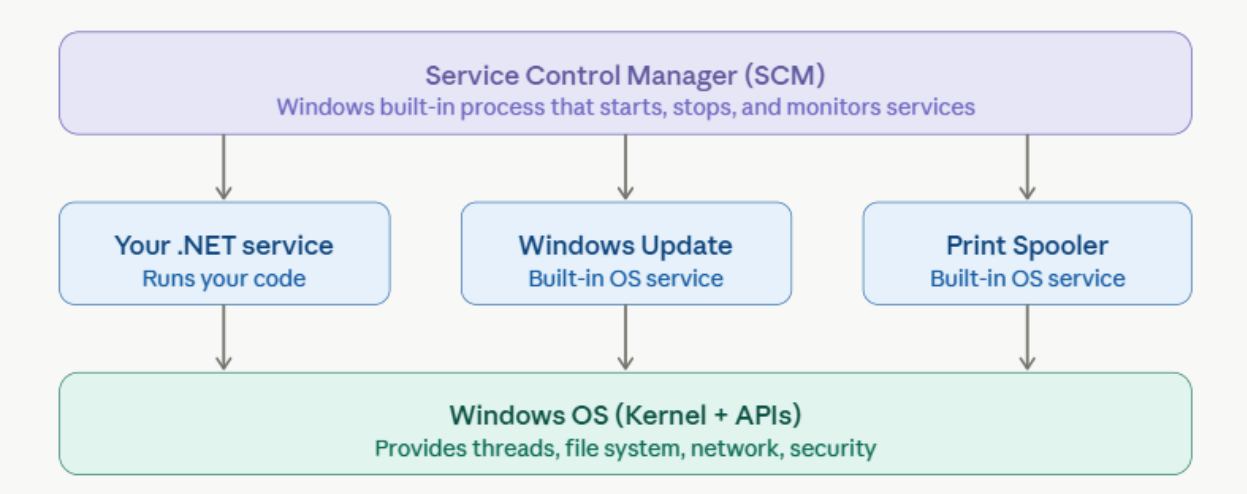
Feature	Explanation
Background execution	No UI
Lifecycle Managed by OS	Start, stop, Pause
Resilience	Can auto-start on failure

When to use Windows Services :

Use when you need

- Continuous processing (polling DB, queus)
- Scheduled background jobs
- Event driven programming (RabbitMQ, Kafka)
- File watchers , ETL pipelines

Core Architecture Components



1. Service Control Manager (SCM)

The SCM is the central authority for all Windows Services. It is a built-in Windows process (services.exe) that manages the service database, controls service lifecycle, and monitors service health.

- Maintains the list of all registered services
- Starts, stops, pauses, and resumes services
- Notifies services of system shutdown
- Enforces security: only authorised accounts can install or control services

2. Service Process

The service process is your actual .exe file. It must register itself with the SCM on startup and respond to SCM control commands. A single .exe can host multiple services.

3. Service Database

All service configuration is stored in the Windows Registry at `HKEY_LOCAL_MACHINE\SYSTEM\CurrentControlSet\Services`. This holds the service name, binary path, startup type, dependencies, and account information.

4. Service Account

Every service runs under a Windows user account. The choice of account controls what the service can access:

Account	When to use it
LocalSystem	Full local access. Avoid in production — too powerful.
NetworkService	Limited local rights, can access network resources.
LocalService	Minimal privileges, no network access. Most secure.
Custom AD Account	Best for enterprise — assign only required permissions.

Service States

A Windows Service can be in one of the following states at any given time:

State	Description
Stopped	The service is not running.
Start Pending	The service is in the process of starting.
Running	The service is running normally.
Pause Pending	The service is in the process of pausing.
Paused	The service is paused.
Continue Pending	The service is resuming from a paused state.
Stop Pending	The service is in the process of stopping.

Startup Types

Type	Behaviour
Automatic	Starts when Windows boots. Use for services always required.
Automatic (Delayed)	Starts after boot completes. Improves boot performance.
Manual	Starts only when explicitly triggered by a user or another service.
Disabled	Cannot be started at all — use to permanently disable a service.

2. Creating a Windows Service in .NET

Prerequisites

- **.NET 6 or later** installed (download from microsoft.com/dotnet)
- **Visual Studio 2022** or VS Code with the C# extension
- **Basic C# knowledge** — classes, methods, async/await

> **Recommended Approach**
> Use the Worker Service template — it is the modern, officially recommended way to create Windows Services in .NET. It builds on top of the Generic Host, giving you dependency injection, logging, configuration, and service lifetime management for free.

Step 1 — Create the Project

1. Open Visual Studio and choose **Create a new project**
2. Search for `Worker Service` and select it
3. Name your project (e.g. `MyBackgroundService`) and click Create

Alternatively, from the command line:

```
dotnet new worker -n MyBackgroundService
cd MyBackgroundService
```

Step 2 — Add the NuGet Package

```
dotnet add package Microsoft.Extensions.Hosting.WindowsServices
```

Step 3 — Configure Program.cs

Program.cs is the entry point. The key call here is UseWindowsService(), which tells the .NET host to register the application with the SCM.

```
var builder = Host.CreateApplicationBuilder(args);

// Tell .NET this should run as a Windows Service
builder.Services.AddWindowsService(options =>
{
    options.ServiceName = "My Background Service";
});

// Register your worker class with DI
builder.Services.AddHostedService<Worker>();
```

```
var app = builder.Build();
await app.RunAsync();
```

Step 4 — Write Your Worker Class

The Worker class is where your background logic lives. You inherit from BackgroundService and override ExecuteAsync, which runs continuously while the service is active.

```
public class Worker : BackgroundService
{
    private readonly ILogger<Worker> _logger;

    public Worker(ILogger<Worker> logger)
    {
        _logger = logger;
    }

    // Called once when the service starts
    protected override async Task ExecuteAsync(
        CancellationToken stoppingToken)
    {
        while (!stoppingToken.IsCancellationRequested)
        {
            _logger.LogInformation("Running at: {time}",
                DateTimeOffset.Now);

            // Your background work goes here
            await Task.Delay(TimeSpan.FromSeconds(10),
                stoppingToken);
        }
    }
}
```

Important: CancellationToken
Always pass the stoppingToken to every async call (Task.Delay, database queries, HTTP requests). When the SCM asks the service to stop, this token is cancelled. If you ignore it, your service may take a long time to shut down — the SCM will eventually force-kill it.

Service Lifecycle Methods

BackgroundService provides three lifecycle hooks you can override:

Method	When it is called
StartAsync(token)	Before ExecuteAsync begins. Good for initialisation work.
ExecuteAsync(token)	Your main loop. Runs until the token is cancelled.
StopAsync(token)	After cancellation. Good for cleanup (close connections, flush logs).

Build and Publish

Before installing, publish a self-contained executable:

```
dotnet publish MyService.csproj -c Release -r win-x64 --self-contained true
-o C:\Services\MyService
```

3. Installing & Managing Windows Services

Run as Administrator
All install, configure, and uninstall commands require an elevated (admin) command prompt or PowerShell window. Right-click the terminal icon and choose 'Run as administrator'.

Method 1 — sc.exe (Built-in Command Line Tool)

sc.exe (Service Control) is available on every Windows system. It is the most direct way to interact with the SCM.

```
# Install the service
sc.exe create "MyService" ^
    binPath= "C:\Services\MyService\MyService.exe" ^
    DisplayName= "My Background Service" ^
    start= auto

# Start the service
sc.exe start "MyService"

# Check the current status
sc.exe query "MyService"

# Stop the service
sc.exe stop "MyService"

# Remove the service
sc.exe delete "MyService"
```

Common Mistake
The spaces after binPath= and start= in sc.exe are mandatory. 'binPath=C:\...' will fail; 'binPath= C:\...' (with a space after =) is correct.

Method 2 — PowerShell (Modern Approach)

PowerShell provides more readable commands and is the preferred approach in modern Windows administration and CI/CD pipelines.

```
# Install the service
New-Service -Name "MyService" `
    -BinaryPathName "C:\Services\MyService.exe" `
    -DisplayName "My Background Service" `
    -StartupType Automatic

# Start / Stop / Restart
Start-Service   -Name "MyService"
Stop-Service    -Name "MyService"
```

```
Restart-Service -Name "MyService"

# Check status
Get-Service -Name "MyService"

# View all services
Get-Service | Sort-Object Status
```

Method 3 — Services Management Console (services.msc)

The GUI approach is beginner-friendly and excellent for configuring recovery options.

4. Press **Win + R**, type `services.msc`, press Enter
5. Find your service. Right-click it and choose **Properties**
6. On the **General** tab: set Startup Type (Automatic / Manual / Disabled)
7. On the **Log On** tab: set the service account
8. On the **Recovery** tab: configure automatic restart on failure

Configuring Recovery Options

Recovery options tell Windows what to do if your service crashes. This is critical in production environments.

Setting	Recommended Value
First failure	Restart the Service
Second failure	Restart the Service
Subsequent failures	Restart the Service
Reset fail count after	1 day
Restart service after	1 minute

You can also set recovery options from PowerShell:

```
sc.exe failure "MyService" reset= 86400 ^
    actions= restart/60000/restart/60000/restart/60000
```

4. Debugging Windows Services

Why Is Debugging Services Tricky?

Services are managed by the SCM — there is no console window, no easy F5-to-run, and startup happens before you can attach a debugger. The SCM also imposes a startup time limit (typically 30 seconds), so if your service hangs during startup while waiting for a debugger, the SCM will kill it.

Approach 1 — Console Mode Detection (Recommended for Beginners)

This is the easiest and most practical approach. You make your code detect whether it is running under the SCM or launched directly, and behave accordingly.

```
var builder = Host.CreateApplicationBuilder(args);

// Only register as a Windows Service if actually running as one
if (WindowsServiceHelpers.IsWindowsService())
{
    builder.Services.AddWindowsService();
}
// Otherwise it runs as a normal console app

builder.Services.AddHostedService<Worker>();
var app = builder.Build();
await app.RunAsync();
```

Why This Works

IsWindowsService() returns false when you press F5 in Visual Studio, so the app runs as a normal console app where all breakpoints and debug tools work perfectly. When installed and started by the SCM, it returns true and behaves as a real service.

Approach 2 — Attach Debugger to Running Service

Use this when you need to debug the actual installed service.

9. Install and start the service as normal (see Section 3)
10. In Visual Studio: **Debug** menu → **Attach to Process** (Ctrl+Alt+P)
11. In the process list, find your service `.exe` name
12. Set **Attach to: Managed (.NET) code**
13. Click **Attach** — your breakpoints will now fire

Gotcha

The service must be built in Debug configuration (not Release) for breakpoints to bind correctly. Release builds are optimised and strip out debugging information.

Approach 3 — Adding a Startup Delay for Attach

If you need to debug service startup code, add a temporary delay so you have time to attach the debugger:

```
protected override async Task ExecuteAsync(
    CancellationToken stoppingToken)
{
    // TEMPORARY: Remove before production!
    await Task.Delay(TimeSpan.FromSeconds(20), stoppingToken);

    // Your normal startup logic here
    while (!stoppingToken.IsCancellationRequested)
    {
        await DoWorkAsync(stoppingToken);
        await Task.Delay(TimeSpan.FromSeconds(10), stoppingToken);
    }
}
```

Approach 4 — Structured Logging to Event Viewer

Even with a debugger, good logging is essential in production. The Windows Event Log is always available and does not require any additional setup.

```
// Program.cs — enable Event Log
builder.Logging.AddEventLog(new EventLogSettings
{
    SourceName = "MyBackgroundService"
});

// Worker.cs — use log levels
_logger.LogInformation("Service started successfully");
_logger.LogWarning("Queue is backing up: {count} items", count);
_logger.LogError(ex, "Failed to process batch {batchId}", batchId);
```

View logs in **Event Viewer**: press Win+R, type `eventvwr.msc`, then navigate to Windows Logs → Application.

Common Errors & Fixes

Error / Symptom	Likely Cause & Fix
Service stops immediately after starting	An unhandled exception in ExecuteAsync. Add a try/catch and log the error.
'Error 1053: service did not respond'	Service took too long to start (>30s). Move heavy init work to ExecuteAsync, not the constructor.
Breakpoints not hit	Service is built in Release mode. Rebuild in Debug mode.
Service starts but does nothing	ExecuteAsync completed immediately. Check your while loop and CancellationToken logic.

'Access is denied' on install	Not running as Administrator. Open terminal with admin rights.

5. Advanced Service Features

Dependency Injection

Because Worker Services use the .NET Generic Host, you get full dependency injection — the same system used in ASP.NET Core. You can inject database contexts, HTTP clients, repositories, and any other service.

```
// Program.cs — register all your services
builder.Services.AddDbContext<AppDbContext>(options =>
    options.UseSqlServer(connectionString));

builder.Services.AddHttpClient<IWeatherClient, WeatherClient>();
builder.Services.AddSingleton<IReportService, ReportService>();
builder.Services.AddHostedService<Worker>();
```

Scoped Services Warning

BackgroundService is a singleton. Injecting a scoped service (like DbContext) directly into the constructor will throw an error at runtime. Instead, inject IServiceScopeFactory and create a new scope for each unit of work.

```
public class Worker : BackgroundService
{
    private readonly IServiceScopeFactory _scopeFactory;

    public Worker(IServiceScopeFactory scopeFactory)
    {
        _scopeFactory = scopeFactory;
    }

    protected override async Task ExecuteAsync(
        CancellationToken stoppingToken)
    {
        while (!stoppingToken.IsCancellationRequested)
        {
            // Create a new DI scope for each work unit
            using var scope = _scopeFactory.CreateScope();
            var db = scope.ServiceProvider
                .GetRequiredService<AppDbContext>();

            await ProcessDataAsync(db, stoppingToken);
            await Task.Delay(TimeSpan.FromMinutes(1), stoppingToken);
        }
    }
}
```

Task Scheduling with PeriodicTimer

PeriodicTimer, introduced in .NET 6, is the cleanest way to run recurring work. Unlike Task.Delay in a loop, it does not drift — each tick fires at the interval boundary regardless of how long your work took.

```
protected override async Task ExecuteAsync(
    CancellationToken stoppingToken)
{
    using PeriodicTimer timer =
        new(TimeSpan.FromMinutes(5));

    while (await timer.WaitForNextTickAsync(stoppingToken))
    {
        await DoWorkAsync(stoppingToken);
    }
}
```

Configuration with appsettings.json

Use appsettings.json for all configurable values (intervals, connection strings, endpoints). Never hardcode these in your service.

```
// appsettings.json
{  "ServiceSettings": {
    "PollingIntervalSeconds": 30,
    "MaxRetries": 3
  }
}

// Register in Program.cs
builder.Services.Configure<ServiceSettings>(
    builder.Configuration.GetSection("ServiceSettings"));

// Inject in Worker
public Worker(IOptions<ServiceSettings> settings)
{
    _settings = settings.Value;
}
```

Multiple Hosted Services

You can register multiple independent workers in one service process. Each runs on its own background thread. This is useful for services that need to perform several unrelated tasks.

```
// Register multiple workers
builder.Services.AddHostedService<DatabaseCleanupWorker>();
builder.Services.AddHostedService<EmailQueueWorker>();
builder.Services.AddHostedService<MetricsReportWorker>();

// All three run concurrently in the same process
```

Security Best Practices

Practice	Why It Matters

Use least-privilege service accounts	LocalSystem has full access to the machine. A custom AD account with only required permissions limits damage if the service is compromised.
Store secrets in Windows DPAPI or Azure Key Vault	Never store passwords or API keys in appsettings.json in plain text.
Validate all external input	Services often process data from queues or files. Validate and sanitise inputs to prevent injection attacks.
Run with a dedicated local or AD account	Allows auditing — all actions are attributed to the service account in security logs.

CI/CD Deployment Pattern

A typical automated deployment to a Windows Server follows this sequence:

14. **Build:** dotnet publish -c Release -r win-x64 --self-contained true
15. **Stop:** Stop-Service -Name "MyService" -Force
16. **Deploy:** Copy new files to the service folder, overwriting old files
17. **Start:** Start-Service -Name "MyService"
18. **Verify:** (Get-Service "MyService").Status — confirm it is Running

First Deployment
On a fresh server, run sc.exe create (or New-Service) once to register the service. Subsequent deployments only need the stop → copy → start steps.

Quick Reference Card

Keep this page handy as a cheat sheet when working with Windows Services.

Essential Commands

Task	Command
Install service	sc.exe create "Name" binPath= "C:\path\to.exe" start= auto
Start service	sc.exe start "ServiceName" or Start-Service -Name "Name"
Stop service	sc.exe stop "ServiceName" or Stop-Service -Name "Name"
Check status	sc.exe query "ServiceName" or Get-Service -Name "Name"
Remove service	sc.exe delete "ServiceName"
Open Services GUI	Win+R → services.msc
Open Event Viewer	Win+R → eventvwr.msc
Open Registry	Win+R → regedit (HKLM\SYSTEM\CurrentControlSet\Services)

Beginner Checklist

- Build in Debug mode when debugging, Release mode for production
- Always pass CancellationToken to every async method
- Use try/catch inside ExecuteAsync and log all exceptions
- Use IServiceScopeFactory for scoped services like DbContext
- Run install/uninstall commands as Administrator
- Configure recovery options in services.msc for production
- Never hardcode connection strings — use appsettings.json or secrets
- Test as a console app first, then install as a service